ARKA-Lite

Proyecto final · Upskilling Backend Java

Equipo arka-litos: dos microservicios Spring Boot, sus bases de datos y su plataforma, contados de punta a punta.

RECORRIDO EJECUTIVO · CONTENIDO Y GUION

PARTES

1. **Fundamentos** Primero el qué, después el cómo: el negocio claro y buenas prácticas que sirven en cualquier lenguaje.
2. **Arquitectura** El sistema por fuera: qué piezas hay y cómo trabajan juntas.
3. **Base de datos** Dos bases PostgreSQL, el ORM, la cadena de conexión y las transacciones.
4. **Interconexión de servicios** Cómo se hablan los servicios: síncrono, asíncrono, Outbox e idempotencia.
5. **DevOps: Docker y Kubernetes** Docker empaqueta, Kubernetes administra y el pipeline prueba cada cambio.
6. **Observabilidad** Logs en un solo lugar, trazas entre servicios y salud de cada componente.
7. **Demostración con la UI** La UI ejemplifica en vivo lo que contamos: estados, Outbox, idempotencia y salud.

Generado desde docs/presentation-data/contenido. Cada lámina tiene: lo que se ve, el diagrama detallado cuando lo hay, y el guion: qué decir, qué hacer, si preguntan y para pasar.

Apertura

1. Del endpoint al clúster

Presenta: Orquestador

EN PANTALLA · NÚMEROS GRANDES

- 2 microservicios
- 2 bases PostgreSQL
- 1 comando levanta todo
- 0 avisos perdidos o duplicados

IDEAS CLAVE

- Cómo construimos dos microservicios con sus datos, su plataforma y su operación.

LO QUE SE LLEVA EL STAFF

En 40 minutos, prácticas que pueden aplicar en sus propios sistemas, no solo teoría.

QUÉ DECIR (? = HACER CLIC)

"Buenos días a todos. Somos el equipo arka-litos, del curso Upskilling Backend Java, y hoy les presentamos ARKA-Lite, nuestro proyecto final."

"El título lo resume: empezamos escribiendo un endpoint, una sola operación en Java, y terminamos con un sistema completo corriendo en un clúster de Kubernetes, con sus bases de datos, su pipeline y su monitoreo."

"El equipo anterior ya les explicó los fundamentos y la arquitectura. Nosotros les vamos a contar el proyecto desde la práctica, con diagramas y una demostración en vivo, y nos vamos a enfocar en lo que les puede servir en su día a día: cómo se cuidan los datos, cómo se hablan los sistemas y cómo se despliega y se opera todo."

QUÉ HACER (NO SE DICE)

- Antes de empezar, abrir la UI (http://localhost:8088, o el puerto 8088 del Codespace) y comprobar que los dos servicios aparecen en verde. Guía del Codespace: docs/operaciones/codespaces.md.
- Tener abiertas en otras pestañas Grafana (http://localhost:3000) y la carpeta k8s/ del repositorio, por si preguntan.
- Cada diagrama se arma con clics: en el texto, ? CLIC marca el momento de hacer clic.

SI HAY TIEMPO, SE PUEDE AGREGAR

- ARKA-Lite nació como el laboratorio del curso: cada tema que veíamos lo aplicábamos al mismo sistema. Por eso creció por capas: primero un endpoint, después la base de datos, la comunicación entre servicios, los contenedores, el pipeline y la observabilidad.
- Todo lo que van a ver está en un solo repositorio, con su documentación, y se levanta con un solo comando. Si a alguien le interesa revisarlo después, con gusto se lo compartimos.

PARA PASAR

“Primero, quiénes somos.”

---

2. Equipo arka-litos

Presenta: Orquestador

EN PANTALLA · EL EQUIPO POR ÁREA

- Orquestación: Conoce el negocio completo: abre, conecta las partes y cierra. (1, de desarrollo). Presenta: Apertura, El negocio, Cierre.
- Desarrollo: Escribieron los servicios, la UI y las pruebas. (4 o 5 personas). Presenta: Fundamentos, Arquitectura, El ORM, Interconexión, Demostración.
- Base de datos: Diseñó las tablas y cómo se conecta el código. (1 persona). Presenta: Base de datos.
- Infraestructura: Empaquetan, despliegan y vigilan el sistema. (2 personas). Presenta: Docker y Kubernetes, CI/CD, Observabilidad.

IDEAS CLAVE

- Desarrollo: 4 o 5 personas
- Base de datos: 1 persona
- Infraestructura: 2 personas
- Una persona de desarrollo orquesta: abre, conecta y cierra

LO QUE SE LLEVA EL STAFF

Un mismo proyecto visto desde las tres áreas que ustedes también tienen.

QUÉ DECIR (? = HACER CLIC)

“Somos un equipo de tres áreas, las mismas que tienen ustedes: desarrollo, base de datos e infraestructura. Cada parte la presenta quien la construyó.”

“Desarrollo escribió los servicios, la interfaz y las pruebas, y va a contar los fundamentos, la arquitectura, el ORM, la comunicación entre servicios y la demostración. Base de datos cuenta cómo se guardan y se cuidan los datos. Infraestructura, cómo se empaqueta, se despliega y se vigila. Y yo, que vengo de desarrollo y conozco el negocio de punta a punta, voy a abrir, conectar las partes y cerrar.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Trabajamos como trabajaría un equipo real: cada cambio en una rama, revisado por otra persona y probado por el pipeline antes de entrar.
- Mezclar áreas nos obligó a ponernos de acuerdo en contratos claros: qué expone cada servicio, qué guarda cada base y cómo se despliega cada pieza.

PARA PASAR

“Este es el recorrido.”

---

3. Qué vamos a ver

Presenta: Orquestador

EN PANTALLA · LA RUTA DE LAS PARTES

- 1. Fundamentos (vista general): Primero el qué, después el cómo: el negocio claro y buenas prácticas que sirven en cualquier lenguaje.

- 2. Arquitectura (vista general): El sistema por fuera: qué piezas hay y cómo trabajan juntas.
- 3. Base de datos (a fondo): Dos bases PostgreSQL, el ORM, la cadena de conexión y las transacciones.
- 4. Interconexión de servicios (a fondo): Cómo se hablan los servicios: síncrono, asíncrono, Outbox e idempotencia.
- 5. DevOps: Docker y Kubernetes (a fondo): Docker empaqueta, Kubernetes administra y el pipeline prueba cada cambio.
- 6. Observabilidad (vista general): Logs en un solo lugar, trazas entre servicios y salud de cada componente.
- 7. Demostración con la UI (en vivo): La UI ejemplifica en vivo lo que contamos: estados, Outbox, idempotencia y salud.

IDEAS CLAVE

- Fundamentos y arquitectura: vista general
- Base de datos: a fondo
- Interconexión de servicios: a fondo
- DevOps con Docker y Kubernetes: a fondo
- Observabilidad y demostración en vivo

LO QUE SE LLEVA EL STAFF

Saber desde el inicio dónde vamos a profundizar y qué se van a llevar.

QUÉ DECIR (? = HACER CLIC)

"Todos los equipos presentamos las mismas secciones, pero cada uno profundiza en algunas para no repetirnos. El equipo anterior se enfocó en los fundamentos y en la arquitectura hexagonal, así que esas dos partes las vamos a ver rápido, en vista general."

"? Donde vamos a fondo es en tres temas que tocan directo el trabajo de ustedes: la base de datos, cómo se comunican los servicios sin perder información, y DevOps, con Docker, Kubernetes y el pipeline. ? Después, una mirada general a la observabilidad, ? y cerramos con una demostración en vivo: vamos a tumbar un servicio a propósito para que vean cómo se comporta el sistema."

"Al final de cada parte les dejamos lo que aprendimos y cómo se puede aplicar en cualquier proyecto, también en los que usan procedimientos almacenados."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Las partes generales son cortas a propósito: el equipo anterior ya las explicó. El tiempo lo dedicamos a lo que más valor les deja: los datos, la comunicación entre sistemas y la operación.
- Al final juntamos lo que pueden aplicar desde mañana, aunque sus sistemas usen procedimientos almacenados.

PARA PASAR

"Empecemos por los fundamentos, y por lo primero: el qué."

1 · Fundamentos

Primero el qué, después el cómo: el negocio claro y buenas prácticas que sirven en cualquier lenguaje.

4. Primero el qué: el negocio en una lámina

Presenta: Orquestador

EN PANTALLA · FLUJO PASO A PASO

- 1. Quien solicita: Registra qué necesita y de qué tipo.
- 2. La envía: Cuando está lista.
- 3. El sistema avisa: A quien debe atenderla.
- 4. Alguien decide: La aprueba o la rechaza.
- Si si se salta un paso en el paso 2: No se permite: Enviar dos veces o decidir sobre un borrador: el sistema explica por qué.

IDEAS CLAVE

- Quien solicita registra la solicitud
- La envía cuando está lista
- El sistema le avisa a quien debe atenderla
- Alguien la aprueba o la rechaza
- Saltarse un paso no se permite

LO QUE SE LLEVA EL STAFF

Pensar primero en el qué y después en el cómo: las reglas claras hacen el código simple.

QUÉ DECIR (? = HACER CLIC)

“Antes de entrar a la tecnología, el aprendizaje más importante de los fundamentos: primero pensamos en el qué y después en el cómo. ¿Qué tiene que pasar, quién lo hace y qué no se puede permitir? Este es el negocio, en una lámina. Alguien registra una solicitud, por ejemplo una incidencia o un pedido de cambio: qué necesita y de qué tipo.”

“? Cuando está lista, la envía. ? En ese momento, el sistema le avisa a quien debe atenderla. ? Y esa persona la aprueba o la rechaza: es la decisión final.”

“? Igual de importante es lo que no se permite: enviar dos veces la misma solicitud, o aprobar una que todavía nadie ha enviado. El sistema no lo hace y explica por qué. Esas son las reglas del negocio, y con ellas claras, el código salió simple: un servicio para las solicitudes y otro para los avisos.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Antes de escribir una línea de código nos hicimos tres preguntas: qué tiene que pasar, quién hace cada paso y qué no se puede permitir. Las respuestas se volvieron las reglas del sistema.
- Por dentro, esos pasos son cuatro estados: borrador, enviada, aprobada y rechazada. La base de datos también los cuida: la columna estado solo acepta esos cuatro valores.
- Al arrancar, el sistema siempre registra dos solicitudes de ejemplo: INC-001, una incidencia en borrador, y CAM-002, un cambio que se envía. El pipeline usa CAM-002 para comprobar que el servicio arrancó de verdad.

SI PREGUNTAN

¿Por qué los avisos van en otro servicio?

Porque cambian por motivos distintos, y así el trámite sigue funcionando aunque el servicio de avisos esté caído.

PARA PASAR

“Con el qué claro, el cómo: código que se pueda cambiar sin miedo. Le paso la palabra a mi compañero.”

---

5. Código mantenible: buenas prácticas

Presenta: Desarrollo 2

EN PANTALLA · TARJETAS

- Una tarea por pieza: Quien recibe la petición no decide; quien decide no guarda.
- Piezas que se conectan: Cada pieza recibe lo que necesita: se cambia o se prueba sola.
- Reglas en un solo lugar: El negocio vive junto, lejos de la pantalla y de la base.
- Errores con respuesta clara: Siempre el mismo formato: 400, 404 o 409 con su motivo.
- Nombres del negocio: Solicitud, enviar, aprobar: el código se lee como el proceso.
- Cada regla con su prueba: Una prueba automática avisa si un cambio rompió algo.

IDEAS CLAVE

- Una tarea por pieza
- Las piezas se conectan, no se fabrican adentro
- Las reglas viven en un solo lugar
- Errores con respuesta clara
- Nombres del negocio
- Cada regla con su prueba

LO QUE SE LLEVA EL STAFF

Cuando cambia una regla, se toca un solo lugar y una prueba confirma que nada se rompió.

QUÉ DECIR (? = HACER CLIC)

“Gracias. Con el qué claro, viene el cómo. El equipo anterior ya cubrió los fundamentos, así que nos quedamos con seis prácticas que nos sirvieron y que se pueden usar en cualquier lenguaje, sea Java, C#, Python o PL/SQL. Primera: cada pieza hace una sola tarea; la que recibe la petición no decide, y la que decide no guarda.”

“? Segunda: las piezas se conectan desde afuera, así se pueden cambiar o probar por separado. ? Tercera: las reglas del negocio viven en un solo lugar, no repetidas en la pantalla y en la base. ? Cuarta: los errores responden siempre igual: un 400 si faltan datos, un 404 si algo no existe, un 409 si la regla no lo permite.”

“? Quinta: los nombres son los del negocio, así el código se lee como el proceso. ? Y sexta: cada regla tiene una prueba automática que avisa si un cambio rompió algo. El resultado: cuando cambia una regla, se toca un solo lugar.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Un ejemplo concreto: el controlador de solicitudes no decide nada; solo convierte el JSON y llama al servicio. Si mañana cambia la regla de envío, se toca la clase Solicitud y su prueba, nada más.

- Lo de las piezas que se conectan desde afuera tiene un nombre: inyección de dependencias. En ARKA-Lite todo llega por el constructor, y por eso en las pruebas cambiamos la base real por una en memoria sin tocar el negocio.

SI PREGUNTAN

¿Esto no hace el código más largo?
Un poco más de piezas, pero cada una es corta y se entiende sola. El costo se recupera en el primer cambio.

PARA PASAR
"Y como somos varios trabajando sobre el mismo código, también necesitamos orden."

---

6. Trabajo en equipo: Git y documentación

Presenta: Desarrollo 2

EN PANTALLA · UNA RAMA DE GIT

- Rama feat/crear-solicitud-por-post sale de main
- Commit: feat: crear solicitud
- Commit: test: 409 si ya existe
- Commit: docs: el nuevo POST
- Vuelve a main: Pull Request: revisión, pruebas en verde y documentación

IDEAS CLAVE

- Una rama corta por tarea
- Cada commit dice qué cambió
- Entra a main solo por Pull Request
- La documentación viaja en el mismo cambio

LO QUE SE LLEVA EL STAFF

Nada llega a la rama principal sin revisión, pruebas y documentación.

QUÉ DECIR (? = HACER CLIC)

"Muy rápido, cómo trabajamos en equipo. Cada tarea se hace en una rama corta, y cada commit dice qué cambió: una funcionalidad, una prueba, la documentación."

"? El cambio entra a la rama principal solo por un Pull Request: otra persona lo revisa, las pruebas tienen que pasar y la documentación se actualiza en ese mismo cambio. La documentación de la API, además, la genera Swagger desde el código, así que nunca queda vieja."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Usamos un formato común para los mensajes de commit: el tipo, dos puntos y qué hace el cambio, por ejemplo: fix: impedir el envío de una solicitud aprobada. Con eso, el historial se lee como una bitácora.
- Nadie sube directo a main, ni siquiera quien orquesta. Eso nos protegió más de una vez de romper algo que otro compañero estaba usando.

PARA PASAR
"Esto es lo que nos llevamos de los fundamentos."

7. Fundamentos: lo que nos llevamos a cualquier proyecto

Presenta: Desarrollo 2

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 6 prácticas que sirven en cualquier lenguaje
- Métrica: 4 respuestas claras: 201, 400, 404 y 409
- Primero el qué, después el cómo: Qué tiene que pasar, quién lo hace y qué no se permite. Valor: Reglas claras antes de elegir la tecnología.
- Separar responsabilidades paga pronto: Pantalla, reglas y datos en piezas distintas. Valor: Un cambio de regla toca un solo lugar.
- Las reglas no dependen de la herramienta: El negocio no sabe si detrás hay una base, una web o un framework. Valor: Se prueba rápido y sobrevive a un cambio de tecnología.
- Todo cambio entra revisado: Ramas cortas, revisión, pruebas y documentación juntas. Valor: Menos sorpresas en producción.

IDEAS CLAVE

- Primero el qué, después el cómo
- Separar responsabilidades paga en el primer cambio
- Las reglas no dependen de la herramienta
- Todo cambio entra revisado

LO QUE SE LLEVA EL STAFF

Cuatro hábitos que bajan el costo de mantener cualquier sistema.

QUÉ DECIR (? = HACER CLIC)

"Lo que nos llevamos a cualquier proyecto. ? Primero el qué, después el cómo: con las reglas del negocio claras, la tecnología se elige mejor y el código sale más simple. ? Separar responsabilidades se paga desde el primer cambio."

"? Las reglas no deben depender de la herramienta: si mañana cambia la base o el framework, las reglas siguen iguales. ? Y ningún cambio entra sin revisión, pruebas y documentación."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Si tuviéramos que elegir una sola práctica para empezar en un sistema existente, sería poner las reglas del negocio en un solo lugar con su prueba. Es la que más rápido devuelve la inversión.

PARA PASAR

"Con esa base, veamos el sistema completo desde arriba."

2 · Arquitectura

El sistema por fuera: qué piezas hay y cómo trabajan juntas.

8. El sistema de un vistazo

Presenta: Desarrollo 3 (8 personas) · Desarrollo 2 (7 personas)

EN PANTALLA · CAPAS DEL SISTEMA

- Experiencia: Persona: Usa el navegador; arka-lite-ui: La pantalla · reenvía /api
- Servicios: servicio-solicitudes: El trámite y sus estados; servicio-notificaciones: Los avisos, sin duplicados
- Datos: db-solicitudes: PostgreSQL 17 · solicitud y evento; db-notificaciones: PostgreSQL 17 · evento_procesado
- Plataforma: Compose o Kubernetes: Dónde corre; GitHub Actions: Prueba y publica; Loki y Grafana: Logs en un solo lugar

IDEAS CLAVE

- La UI es la puerta de entrada
- Dos servicios, cada uno dueño de su base
- Entre servicios solo viajan datos
- La plataforma corre, entrega y observa

LO QUE SE LLEVA EL STAFF

Cada capa se puede cambiar sin romper las otras.

QUÉ DECIR (? = HACER CLIC)

"Gracias. ARKA-Lite no es una aplicación grande: son piezas pequeñas, cada una con una responsabilidad. Arriba está la experiencia: la persona usa la interfaz, y la interfaz no guarda nada, solo les pasa las peticiones a los servicios."

"? En el medio, los dos microservicios: solicitudes lleva el trámite y sus estados; notificaciones crea los avisos. ? Debajo, cada uno con su propia base PostgreSQL; ninguno lee la base del otro. ? Y abajo, la plataforma: dónde corre, cómo se prueba cada cambio y dónde se ven los logs."

SI HAY TIEMPO, SE PUEDE AGREGAR

- La interfaz es delgada a propósito: no guarda nada ni toma decisiones. Si mañana se cambia por otra, los servicios no se enteran.
- La plataforma es la misma para los dos servicios: el mismo Compose, el mismo pipeline y el mismo lugar para los logs. Eso baja mucho el costo de tener dos servicios en lugar de uno.

SI PREGUNTAN

¿Por qué no comparten una sola base de datos?

Para que cada servicio sea dueño de sus datos y pueda cambiar sin romper al otro.

PARA PASAR

"Ahora veamos cómo trabajan juntas esas piezas en la práctica."

9. Cómo funciona en la práctica

Presenta: Desarrollo 3 (8 personas) · Desarrollo 2 (7 personas)

EN PANTALLA · EL SISTEMA EN LA PRÁCTICA

- 1. Persona -> La UI: usa
- 2. La UI -> Solicitudes: crea, envía y decide

- 3. Solicitudes -> db-solicitudes: guarda
- 4. Solicitudes ? Notificaciones: avisa después (SolicitudEnviada: eventoId, id y tipo)
- 5. Notificaciones -> db-notificaciones: anota lo recibido
- 6. La UI -> Notificaciones: consulta los avisos
- Ninguna flecha une las dos bases: cada servicio es dueño de sus datos.

IDEAS CLAVE

- La persona usa la UI
- La UI le pide el trabajo a cada servicio
- Cada servicio guarda en su propia base
- Solicitudes avisa con un evento: un dato, no una pieza

LO QUE SE LLEVA EL STAFF

Los sistemas se comunican con mensajes claros, no compartiendo tablas.

QUÉ DECIR (? = HACER CLIC)

“Ahora, cómo trabajan juntas en la práctica. Sigamos los números. ? Uno: la persona usa la interfaz. ? Dos: cuando crea, envía o decide una solicitud, la interfaz se lo pide al servicio de solicitudes. ? Tres: solicitudes lo guarda en su propia base.”

“? Cuatro, el paso más importante: cuando una solicitud se envía, solicitudes le avisa a notificaciones. Fíjense en que esa flecha no es una pieza del sistema: es un dato que viaja, un mensaje pequeño con el número del evento, la solicitud y su tipo.”

“? Cinco: notificaciones crea el aviso y anota que ya recibió ese evento. ? Seis: la interfaz le consulta a notificaciones para mostrar los avisos.”

“? Y lo que no hay también dice mucho: ninguna flecha une las dos bases. Si un servicio necesita algo del otro, se lo pide; nunca entra a su base.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- El mensaje que viaja entre servicios lleva solo tres datos: el número del evento, el identificador de la solicitud y su tipo. Mientras más pequeño el mensaje, más fácil es que los dos lados se pongan de acuerdo.
- Ese contrato está copiado en los dos servicios. Si un día cambia, se cambia en los dos al mismo tiempo: es lo único que comparten.

SI PREGUNTAN

¿Qué pasa si notificaciones está caído en el paso cuatro?
Solicitudes sigue funcionando y el aviso espera guardado. Lo vemos en la parte de interconexión y en la demostración.

PARA PASAR

“¿Y cómo está ordenado cada servicio por dentro?”

10. Por dentro de cada servicio: hexagonal

Presenta: Desarrollo 3 (8 personas) · Desarrollo 2 (7 personas)

EN PANTALLA · PUERTOS Y ADAPTADORES

- Centro: Reglas del negocio: La solicitud y sus estados
- Puertos: Guardar solicitudes, Anotar el evento, Avisar a otros
- Entrada: API REST: La web entra aquí; Errores claros: 400 · 404 · 409
- Salida: PostgreSQL: Con el ORM; Bandeja de salida: Tabla evento; HTTP: Hacia notificaciones; En memoria: Para las pruebas

IDEAS CLAVE

- Las reglas del negocio al centro
- La web entra por un lado
- La base y los otros servicios salen por el otro
- Por fuera sigue siendo un servicio en su contenedor

Comparación: Como un enchufe: el aparato funciona con cualquier toma que cumpla la norma, sin saber de qué planta viene la electricidad.

LO QUE SE LLEVA EL STAFF

Cambiar la base o la forma de avisar sin tocar las reglas del negocio.

QUÉ DECIR (? = HACER CLIC)

"Como explicó el equipo anterior, la arquitectura hexagonal es una forma de ordenar el código por dentro. No cambia cómo se despliega: por fuera, cada servicio sigue siendo una aplicación en su contenedor. Al centro van las reglas del negocio y los contratos que necesitan, como guardar una solicitud o avisar que se envió."

"? Por un lado entra la web. ? Por el otro salen la base de datos y las llamadas a otros servicios, conectadas como enchufes. Fíjense en la última: una versión en memoria que reemplaza a la base en las pruebas. Gracias a eso probamos todas las reglas sin levantar PostgreSQL."

SI HAY TIEMPO, SE PUEDE AGREGAR

- En solicitudes hay tres contratos hacia afuera: guardar solicitudes, anotar eventos y publicarlos. Cada uno puede tener más de una implementación, y Spring usa la que está marcada como principal.
- Dejamos a propósito implementaciones alternativas, por ejemplo un publicador simple sin reintentos, para mostrar que se puede cambiar una por otra sin tocar las reglas.

SI PREGUNTAN

¿Esto se parece a llamar un procedimiento almacenado?

En que quien llama solo conoce el nombre y los parámetros, no cómo está hecho por dentro. La diferencia es que aquí el negocio define el contrato y la base se adapta a él.

PARA PASAR

"Esto es lo que nos dejó la arquitectura."

11. Arquitectura: lo que aprendimos

Presenta: Desarrollo 3 (8 personas) · Desarrollo 2 (7 personas)

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 2 servicios, cada uno con su base
- Métrica: 0 tablas compartidas entre servicios
- Cada servicio, dueño de sus datos: Nadie lee ni escribe en la base del otro. Valor: Cada uno cambia sin romper al otro.
- Entre servicios viajan datos: Un evento pequeño con su número: SolicitudEnviada. Valor: Un acuerdo claro y fácil de revisar.
- Hexagonal ordena por dentro: No cambia el despliegue: un contenedor por servicio. Valor: Cambiar la base o la web sin tocar las reglas.
- Separar tiene un costo: Más red, dos bases y fallas parciales. Valor: Separar solo cuando el negocio lo pide.

IDEAS CLAVE

- Cada servicio es dueño de sus datos
- Entre servicios viajan datos, no tablas
- Hexagonal ordena por dentro, no cambia el despliegue
- Separar tiene un costo

LO QUE SE LLEVA EL STAFF

Sistemas que cambian por separado, unidos por acuerdos claros.

QUÉ DECIR (? = HACER CLIC)

“Cuatro aprendizajes de arquitectura. ? Cada servicio es dueño de sus datos. ? Entre servicios no se comparten tablas, se comparten mensajes.”

“? Hexagonal ordena el código por dentro, pero no cambia el despliegue. ? Y separar en microservicios tiene un costo real, así que se hace cuando el negocio lo justifica, no por moda.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Lo más difícil de los microservicios no fue el código, fueron las fronteras: decidir qué dato es de quién. Una vez claro eso, lo demás se ordenó solo.

PARA PASAR

“Y aquí empieza la parte en la que vamos a fondo: la base de datos.”

3 · Base de datos

Dos bases PostgreSQL, el ORM, la cadena de conexión y las transacciones.

12. Dos bases, una por servicio

Presenta: Base de datos

EN PANTALLA · NÚMEROS GRANDES

- 2 bases de datos: db-solicitudes y db-notificaciones: una por servicio.
- 17 PostgreSQL: El mismo motor y la misma versión en las dos.

- 0 JOIN entre bases: Ni consultas ni llaves foráneas que crucen de una a otra.
- 1 disco por base: En Kubernetes, los datos sobreviven aunque el contenedor se reinicie.

IDEAS CLAVE

- Dos bases de datos, una por servicio
- El mismo motor: PostgreSQL 17
- Ninguna consulta cruza de una base a otra
- En Kubernetes, cada base tiene su disco

LO QUE SE LLEVA EL STAFF

Cada base cambia a su ritmo, sin coordinar despliegues con otros equipos.

QUÉ DECIR (? = HACER CLIC)

"Gracias. Como les contó mi compañero de arquitectura, el sistema tiene dos bases de datos, una para cada microservicio: db-solicitudes y db-notificaciones. ? Las dos están montadas en PostgreSQL, versión diecisiete."

"¿Por qué dos y no una? Porque cada servicio es dueño de sus datos. Solicitudes guarda las solicitudes y los avisos que tiene pendientes; notificaciones guarda qué avisos ya recibió. Si mañana una cambia su estructura, la otra ni se entera."

"? Eso tiene una consecuencia que para quienes trabajamos con bases de datos pesa mucho: no hay ningún JOIN entre las dos bases, ni llaves foráneas que crucen. Si un servicio necesita algo del otro, se lo pregunta por HTTP."

"? Y cada base corre en su propio contenedor. En Kubernetes, además, tiene su disco persistente: los datos sobreviven aunque el contenedor se reinicie."

SI HAY TIEMPO, SE PUEDE AGREGAR

- En nuestra computadora, la base de notificaciones se publica en el puerto 5433 para consultarla con cualquier herramienta; la de solicitudes solo se ve dentro de la red de Docker. Lo que no se publica, no se alcanza desde afuera.
- En Kubernetes, cada base tiene su disco persistente: si el contenedor se reinicia, los datos siguen ahí. Solo se perderían si se borra el disco.

SI PREGUNTAN

¿Y si necesito un reporte que junte datos de las dos?

Se arma fuera de los servicios, por ejemplo con una réplica o un proceso de reportes que lea de ambas. Lo que no se hace es que un servicio lea la base del otro.

SI PREGUNTAN

¿No es más difícil administrar dos bases?

Son dos bases pequeñas con una sola responsabilidad cada una. Se respaldan y se monitorean como cualquier PostgreSQL, y la caída de una no tumba a la otra.

PARA PASAR

"Veamos qué hay dentro de cada una."

13. La estructura de las bases

Presenta: Base de datos

EN PANTALLA · BASES DE DATOS Y SUS TABLAS

- db-solicitudes (PostgreSQL 17 · schema.sql): solicitud [id · PK · VARCHAR(20), tipo · VARCHAR(20), estado · CHECK: 4 estados]; evento [id · PK · BIGSERIAL, solicitud_id · FK a solicitud, tipo · SolicitudEnviada, evento_id · UUID, procesado · false = pendiente]
- db-notificaciones (PostgreSQL 17 · Hibernate): evento_procesado [evento_id · PK, uno por evento recibido]
- Entre bases: evento_id por HTTP

IDEAS CLAVE

- solicitud: el trámite y su estado
- evento: los avisos pendientes de entregar
- evento_procesado: los avisos ya recibidos
- Lo único que cruza es el evento_id, por HTTP

LO QUE SE LLEVA EL STAFF

Un modelo pequeño, con reglas que también cuida la base: CHECK, llaves e índices.

QUÉ DECIR (? = HACER CLIC)

"Veamos qué hay adentro. En la base de solicitudes, la tabla solicitud guarda el trámite: su identificador como llave primaria, el tipo y el estado. Y el estado tiene un CHECK que solo admite los cuatro valores válidos. Aunque alguien escriba directo en la tabla, la base no acepta un estado inventado."

"? Al lado está la tabla evento, que funciona como bandeja de salida: cada fila es un aviso que hay que entregar. Tiene una llave foránea a la solicitud, un número único de evento, que es un UUID, y una columna procesado: mientras está en falso, el aviso sigue pendiente."

"? En la base de notificaciones hay una sola tabla importante: evento_procesado, la lista de eventos que ya se recibieron. Su llave primaria es justamente ese número de evento."

"? Y fíjense en la flecha: lo único que pasa de una base a la otra es el evento_id, y no viaja por la base, viaja por HTTP. Esa es la frontera entre los dos servicios."

SI HAY TIEMPO, SE PUEDE AGREGAR

- La tabla evento tiene un índice parcial: solo indexa las filas pendientes. Como el relevador busca pendientes cada cinco segundos, el índice se mantiene pequeño aunque la tabla acumule años de eventos ya entregados.
- Las tablas se crean con un archivo schema.sql que corre en cada arranque y se puede ejecutar mil veces sin romper nada. También actualiza bases viejas, agregando lo que les falte.

SI PREGUNTAN

¿Dónde se guarda el texto de cada aviso?

En este proyecto, en la memoria del servicio de notificaciones; en la base solo queda qué eventos se procesaron. En producción sería una tabla más.

SI PREGUNTAN

¿Por qué un UUID y no un número secuencial?

Porque lo genera la aplicación sin consultar a la base y no se repite entre servicios. Así el mismo número sirve para reintentar sin duplicar.

PARA PASAR

“Ahora, la gran diferencia con muchas aplicaciones que manejamos hoy: dónde vive la regla del negocio.”

---

14. Del procedimiento almacenado al ORM

Presenta: Base de datos

EN PANTALLA · DOS CAMINOS, LADO A LADO

- Hoy: procedimientos almacenados: Pantalla: La persona pulsa Enviar. -> [clic] Aplicación: Solo pasa los parámetros. -> [EXEC sp_enviar @id] Base de datos: El procedimiento decide y guarda. (aquí vive la regla)
- ARKA-Lite: un ORM: Pantalla: La persona pulsa Enviar. -> [POST /enviar] Servicio Java: Decide, con su prueba. (aquí vive la regla) -> [save(…)] Hibernate: Escribe el SQL. -> [UPDATE + INSERT] PostgreSQL: Guarda y cuida llaves e índices.

IDEAS CLAVE

- Con SP, la regla vive en la base
- Con ORM, la regla vive en el código
- Hibernate escribe el SQL de todos los días
- La base sigue cuidando datos, llaves e índices

LO QUE SE LLEVA EL STAFF

Reglas que se prueban en segundos y una base que sigue cuidando los datos.

QUÉ DECIR (? = HACER CLIC)

“Esta es la gran diferencia con muchas de las aplicaciones que gestionamos hoy, y no es la base de datos: es dónde vive la regla del negocio. Arriba, el enfoque que conocemos: la persona pulsa enviar, la aplicación solo pasa parámetros con un EXEC, y el procedimiento almacenado decide y guarda. La regla vive en la base.”

“? Abajo, ARKA-Lite: la regla vive en el servicio Java, junto con su prueba. Cuando decide que sí, Hibernate, que es el ORM, escribe el UPDATE y el INSERT. La base sigue guardando y cuidando llaves, restricciones e índices. Lo único que se movió es la decisión.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- No estamos diciendo que los procedimientos estén mal: son la herramienta correcta para procesos masivos y reportes. Lo que cambia es dónde ponemos las reglas que más cambian y que más necesitamos probar.
- Una ventaja que no se ve a primera vista: con la regla en el código, cada cambio pasa por revisión, pruebas automáticas y pipeline, igual que cualquier otra línea del sistema.

SI PREGUNTAN

¿Y el rendimiento? ¿El SQL que genera Hibernate es bueno?

Para guardar y buscar por llave, sí. Para consultas complejas se puede escribir SQL propio; además, los índices y las restricciones siguen siendo trabajo de la base.

SI PREGUNTAN

¿Hay que migrar todos los procedimientos?

No. Se puede empezar por las reglas que más cambian; los reportes y los procesos masivos pueden seguir en la base.

PARA PASAR

“Veámoslo con una regla real, escrita de las dos formas.”

---

15. La misma regla, escrita de las dos formas

Presenta: Base de datos

EN PANTALLA · LA MISMA IDEA, EN DOS CÓDIGOS

- Hoy: en un procedimiento: CREATE PROCEDURE sp_enviar @id VARCHAR(20) AS BEGIN IF (SELECT estado FROM solicitud WHERE id = @id) <> 'BORRADOR' THROW 50001, 'Solo se envía en BORRADOR', 1; UPDATE solicitud SET estado = 'ENVIADA' WHERE id = @id; END · La regla y el dato viven juntos en la base; Probarla exige una base con datos; Cambiarla es desplegar en la base
- ARKA-Lite: en el código: public Solicitud enviar() { if (estado != Estado.BORRADOR) throw new TransicionNoPermitida("solo se puede enviar en BORRADOR"); return new Solicitud(id, tipo, Estado.ENVIADA); } // Hibernate escribe el SQL: // UPDATE solicitud SET estado = ? WHERE id = ? · La regla vive en el código, con su prueba; Se prueba en milisegundos, sin base; La base sigue cuidando llaves, CHECK e índices

IDEAS CLAVE

- Hoy: la regla dentro de un procedimiento almacenado
- ARKA-Lite: la misma regla en un método Java
- El SQL rutinario lo escribe Hibernate
- La base sigue cuidando llaves, CHECK e índices

LO QUE SE LLEVA EL STAFF

La misma regla, más fácil de probar y de cambiar, sin que la base pierda su papel.

QUÉ DECIR (? = HACER CLIC)

“Tomemos una regla real del negocio: solo se puede enviar una solicitud que está en borrador. A la izquierda, cómo la escribiríamos hoy: un procedimiento que consulta el estado, lanza un error si no está en borrador y hace el UPDATE. La regla y el dato viven juntos en la base, y para probarla hace falta una base con datos.”

“? A la derecha, la misma regla en ARKA-Lite: el método enviar de la clase Solicitud. Si el estado no es borrador, lanza un error; si lo es, devuelve la solicitud enviada. No hay SQL: el UPDATE lo escribe Hibernate cuando el servicio guarda, siempre con parámetros.”

“¿Qué ganamos? Esa regla se prueba en milisegundos, sin base de datos, y si cambia, se cambia en un solo lugar junto con su prueba. ¿Qué no perdemos? La base sigue cuidando lo suyo: la llave primaria, el CHECK de los estados y los índices.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- La prueba de esa regla es así de simple: se crea una solicitud ya enviada, se intenta enviarla otra vez y se verifica que el sistema lo rechaza. Corre en milisegundos y no necesita ninguna base.

- Para probar lo mismo en el procedimiento habría que preparar datos en una base, ejecutarlo y revisar el resultado. Funciona, pero es más lento y más difícil de repetir en cada cambio.

SI PREGUNTAN

¿Y si la regla necesita consultar muchos datos?

Para decidir, el servicio pide solo lo que necesita por llave. Si es un cálculo sobre millones de filas, ese trabajo puede seguir en la base, como una consulta o un procedimiento bien delimitado.

PARA PASAR

“Mi compañero de desarrollo les va a mostrar cómo hace Hibernate esa traducción.”

---

16. El ORM por dentro: una clase es una tabla

Presenta: Desarrollo 4 (8 personas) · Desarrollo 3 (7 personas)

EN PANTALLA · DE LA CLASE A LA TABLA

- SolicitudEntity (Clase Java · @Entity · @Table("solicitud")) <-> solicitud (Tabla en PostgreSQL)
- @Id String id <-> id (VARCHAR(20) · PK)
- String tipo <-> tipo (VARCHAR(20))
- String estado <-> estado (VARCHAR(20) · CHECK)
- jpa.save(nueva) -> INSERT INTO solicitud (id, tipo, estado) VALUES (?, ?, ?)
- jpa.findById("CAM-002") -> SELECT id, tipo, estado FROM solicitud WHERE id = ?
- jpa.save(enviada) -> UPDATE solicitud SET tipo = ?, estado = ? WHERE id = ?

IDEAS CLAVE

- @Entity dice que la clase es una tabla
- Cada campo es una columna
- @Id marca la llave primaria
- save y findById se vuelven SQL con parámetros

LO QUE SE LLEVA EL STAFF

El SQL de todos los días sale solo y siempre con parámetros: menos código y menos riesgo de inyección.

QUÉ DECIR (? = HACER CLIC)

“Gracias. Esta es la traducción, lado a lado. A la izquierda, una clase Java: SolicitudEntity. Con la anotación arroba Entity le decimos a Hibernate que esta clase representa una tabla, y con arroba Table, cuál: la tabla solicitud. ? Cada campo de la clase corresponde a una columna, y arroba Id marca la llave primaria.”

“Con eso, Hibernate ya sabe todo. Abajo ven qué pasa cuando el código llama a un método. ? Guardar una solicitud nueva se convierte en un INSERT. ? Buscar por identificador, en un SELECT con su WHERE. ? Y guardar una que ya existe, por ejemplo cuando se envía, en un UPDATE.”

“Fíjense en los signos de interrogación: los valores viajan siempre como parámetros, nunca pegados al texto del SQL. Es lo mismo que hacemos con los parámetros de un procedimiento, y es lo que nos protege de la inyección SQL.”

“Nosotros no escribimos ninguna de esas consultas. Lo que sí cuidamos fue el diseño de las tablas, porque el ORM confía en que la base esté bien hecha.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- El repositorio de Spring Data no tiene ni una consulta escrita: hereda de JpaRepository, y con eso ya trae guardar, buscar por llave, listar y borrar.
- Cuando el código guarda una solicitud que ya existe, Hibernate primero la busca por su llave y después decide si hace un INSERT o un UPDATE. Por eso las llaves bien definidas importan: el ORM las usa todo el tiempo.

SI PREGUNTAN

¿Se puede ver el SQL que genera?

Sí: con spring.jpa.show-sql=true se imprime en el log. Es lo primero que conviene activar para revisar el rendimiento.

SI PREGUNTAN

¿Y las consultas complejas?

Se pueden escribir a mano, en JPQL o en SQL nativo, con la anotación @Query. El ORM no lo impide.

PARA PASAR

“Ahora sigamos un dato completo, desde la pantalla hasta la fila.”

---

17. El viaje de un dato: de la pantalla a la fila

Presenta: Desarrollo 4 (8 personas) · Desarrollo 3 (7 personas)

EN PANTALLA · FLUJO PASO A PASO

- 1. Pantalla: Manda JSON: SOL-100, tipo Consulta.
- 2. Controlador: Convierte el JSON en un objeto.
- 3. Reglas: Revisa que el id no exista. Sin SQL.
- 4. Hibernate: Objeto -> INSERT con parámetros.
- 5. PostgreSQL: Guarda la fila y hace COMMIT.

IDEAS CLAVE

- La pantalla manda JSON
- El controlador lo convierte en un objeto
- Las reglas validan, sin SQL
- Hibernate arma el INSERT
- PostgreSQL guarda la fila y confirma

Comparación: Como un traductor: el código habla Java, la base habla SQL, y el ORM traduce en los dos sentidos.

LO QUE SE LLEVA EL STAFF

Cada pieza hace una sola traducción; si algo falla, se sabe en qué punto.

QUÉ DECIR (? = HACER CLIC)

“Ahora sigamos un dato de punta a punta. La persona llena el formulario y la pantalla manda un JSON: identificador SOL-100, tipo Consulta. ? El controlador recibe ese JSON y lo convierte en un objeto de Java.”

“? Ese objeto pasa por las reglas del negocio, que revisan, por ejemplo, que el identificador no exista. Hasta aquí no hay ni una línea de SQL. ? Cuando las reglas dicen que sí, Hibernate lo traduce a un INSERT con sus parámetros. ? El driver toma una conexión del pool, y PostgreSQL guarda la fila y confirma con un COMMIT.”

“Y de regreso, el mismo camino al revés: la fila se vuelve objeto, el objeto se vuelve JSON y la pantalla muestra la solicitud creada, con un 201.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Entre el controlador y la base hay dos objetos distintos a propósito: la Solicitud del negocio, que no sabe nada de tablas, y la SolicitudEntity, que es el reflejo de la tabla. Un adaptador traduce entre los dos.
- Si el identificador ya existe, el viaje se corta en las reglas: no se toca la base y la persona recibe un 409 con el motivo.

PARA PASAR

“Para hablar con la base, la aplicación necesita su dirección, su usuario y su clave. Le devuelvo la palabra a mi compañero de base de datos.”

---

18. ¿Dónde está la cadena de conexión?

Presenta: Base de datos

EN PANTALLA · DE DÓNDE SALE CADA VALOR

- Centro: El servicio al arrancar: application.properties solo tiene marcadores. (url=${DB_URL}; username=${DB_USER}; password=${DB_PASSWORD})
- Viene de Docker Compose: compose.yaml, en cada servicio: DB_URL -> db-solicitudes:5432
- Viene de Kubernetes · Deployment: La dirección y el usuario: DB_URL · DB_USER
- Viene de Kubernetes · Secret: La clave, guardada aparte: DB_PASSWORD <- Secret arka-postgres
- Llega a db-solicitudes: PostgreSQL 17 por pool de conexiones
- Nunca: La imagen Docker: no lleva ni dirección ni clave. La misma imagen sirve en cualquier ambiente.

IDEAS CLAVE

- El código solo tiene marcadores: ${DB_URL}
- Compose los llena con variables de entorno
- Kubernetes: la dirección en el Deployment y la clave en un Secret
- La imagen no lleva ni dirección ni clave

LO QUE SE LLEVA EL STAFF

Las claves fuera del código: rotarlas o cambiar de servidor no requiere recompilar.

QUÉ DECIR (? = HACER CLIC)

“Gracias. Esta es la pregunta que siempre nos hacen: ¿dónde está la cadena de conexión? Empecemos por el centro: el servicio, al arrancar, lee su archivo de configuración, y ese archivo no tiene la dirección ni la clave. Solo tiene marcadores: DB_URL, DB_USER y DB_PASSWORD.”

“¿Quién llena esos marcadores? El ambiente donde corre. ? En Docker Compose, las variables se definen para cada servicio en el archivo compose.yaml. ? En Kubernetes, la dirección y el usuario van en el Deployment, ? y la clave se guarda aparte, en un Secret: un objeto del clúster hecho para datos sensibles, que el contenedor recibe como variable al arrancar.”

“? Con esos valores, el servicio abre su pool de conexiones contra la base.”

“? ¿Y la imagen Docker? No lleva ni dirección ni clave. Es la misma imagen en desarrollo, en pruebas y en producción. Cambiar de servidor de base de datos o rotar una clave no obliga a recompilar ni a tocar el código.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Un detalle honesto: el archivo de configuración trae un valor por defecto para trabajar en la computadora de cada uno. En un ambiente real nunca se usa, porque ahí siempre llegan las variables.
- En el pipeline, la clave de la base de pruebas cambia en cada ejecución: se arma con el número de la corrida. Ni siquiera la base temporal tiene una clave fija.

SI PREGUNTAN

¿El Secret no está en el repositorio?

En este laboratorio sí, con una clave de ejemplo, para que cualquiera lo levante. En un ambiente real el Secret se crea aparte, o lo alimenta un gestor de secretos, y nunca se versiona.

SI PREGUNTAN

¿Y si no usamos Kubernetes?

La misma idea funciona con variables de entorno del servidor o con un gestor de secretos. Lo importante es que la clave no viva en el código ni en el paquete que se despliega.

SI PREGUNTAN

¿Quién puede ver la clave en Kubernetes?

Quien tenga permiso de leer Secrets en ese espacio del clúster; ese acceso se controla con roles.

PARA PASAR

“Con la conexión resuelta, veamos qué pasa cuando se guarda: la transacción.”

---

19. Una transacción: todo o nada

Presenta: Base de datos

EN PANTALLA · FLUJO PASO A PASO

- 1. Pool: HikariCP presta una de sus conexiones.
- 2. BEGIN: Empieza la transacción.
- 3. UPDATE: La solicitud pasa a ENVIADA.
- 4. INSERT: El aviso queda pendiente en evento.
- 5. COMMIT: Las dos cosas quedan guardadas.
- Si si algo falla en el paso 4: ROLLBACK: No queda nada a medias: ni la solicitud ni el aviso.

IDEAS CLAVE

- El pool presta una conexión
- BEGIN: empieza la transacción
- Se actualiza la solicitud y se anota el aviso
- COMMIT: todo queda guardado
- Si algo falla: ROLLBACK de todo

LO QUE SE LLEVA EL STAFF

Lo que cambia junto se guarda junto: nunca quedan datos a medias.

QUÉ DECIR (? = HACER CLIC)

"Para quienes administran bases de datos, esto es terreno conocido, solo que configurado desde la aplicación. Cuando llega una petición, el pool de conexiones, HikariCP, le presta una de sus conexiones abiertas; por defecto mantiene hasta diez y las reutiliza."

"? Cuando un método lleva la anotación Transactional, empieza una transacción: es el BEGIN de siempre. ? Al enviar una solicitud, primero se actualiza su estado a enviada, ? y en la misma transacción se anota el aviso pendiente en la tabla evento. ? Al final, el COMMIT guarda las dos cosas juntas."

"? Y si algo falla en el medio, se hace ROLLBACK de todo: no queda una solicitud enviada sin su aviso, ni un aviso sin su solicitud. Esa garantía es la base de lo que les va a explicar mi compañero de interconexión."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Abrir una conexión nueva a PostgreSQL toma su tiempo; con muchas peticiones, eso se nota. Por eso el pool las mantiene abiertas y las presta: es la diferencia entre responder rápido y saturar la base.
- La transacción envuelve todo el método enviar: si el INSERT del evento fallara, por ejemplo por una restricción, el UPDATE de la solicitud también se deshace.

SI PREGUNTAN

¿Cuántas conexiones abre cada servicio?

Las que define el pool; por defecto HikariCP mantiene hasta 10 y las reutiliza entre peticiones.

SI PREGUNTAN

¿Esto reemplaza las transacciones dentro de un procedimiento?

Es el mismo concepto: BEGIN, COMMIT y ROLLBACK. Solo que la transacción la abre la aplicación, alrededor del método que hace el trabajo.

PARA PASAR

"Esto es lo que nos dejó la base de datos."

---

20. Base de datos: lo que aprendimos

Presenta: Base de datos

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 3 tablas en dos bases
- Métrica: 0 consultas que crucen de una base a otra
- El ORM no elimina el trabajo de base de datos: Escribe el SQL rutinario; índices, CHECK y transacciones siguen siendo diseño. Valor: Reglas probadas en código y una base que se cuida sola.

- La cadena de conexión no vive en el código: Variables en Compose y un Secret en Kubernetes. Valor: Cambiar de servidor o de clave sin recompilar.
- La transacción local es la más confiable: Lo que cambia junto se guarda en el mismo COMMIT. Valor: Nunca quedan datos a medias.
- El esquema también es código: schema.sql versionado, idempotente y validado al arrancar. Valor: Despliegues repetibles y errores visibles al inicio.

IDEAS CLAVE

- El ORM no elimina el trabajo de base de datos
- La cadena de conexión no vive en el código
- La transacción local es la más confiable
- El esquema también es código

LO QUE SE LLEVA EL STAFF

El conocimiento de base de datos del staff sigue siendo clave, ahora junto al código.

QUÉ DECIR (? = HACER CLIC)

"Cuatro aprendizajes de datos, sobre todo para quienes venimos de los procedimientos almacenados. ? Primero: el ORM no elimina el trabajo de base de datos. Escribe el SQL rutinario, pero las llaves, las restricciones, los índices y las transacciones siguen siendo diseño de base."

"? Segundo: la cadena de conexión no vive en el código; vive en el ambiente. ? Tercero: la transacción local sigue siendo la herramienta más confiable que tenemos. ? Y cuarto: el esquema también es código: versionado, repetible y validado al arrancar."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Para quien trabaja en base de datos, el rol no desaparece: pasa de escribir cada consulta a diseñar el modelo, cuidar índices y restricciones, y revisar el SQL que genera el ORM.

PARA PASAR

"Esa transacción local es justo la base de la siguiente parte: cómo se avisan los servicios sin perder ni duplicar nada."

4 · Interconexión de servicios

Cómo se hablan los servicios: síncrono, asíncrono, Outbox e idempotencia.

21. Quién habla con quién, y cómo

Presenta: Desarrollo 5 (8 personas) · Desarrollo 4 (7 personas)

EN PANTALLA · TABLA DE TRES COLUMNAS

- Conexión: Navegador -> UI · Cómo se habla: HTTP a localhost:8088; nginx sirve la página · Por qué así: El navegador nunca conoce las direcciones internas
- Conexión: UI -> servicios · Cómo se habla: nginx reenvía /api/solicitudes y /api/notificaciones · Por qué así: Cambiar una dirección no obliga a tocar la UI
- Conexión: solicitudes -> notificaciones · Cómo se habla: POST del evento SolicitudEnviada en JSON, desde el relevador · Por qué así: Si notificaciones cae, solicitudes sigue funcionando
- Conexión: Cómo se encuentran · Cómo se habla: Por nombre: la red de Compose o un Service de Kubernetes · Por qué así: Sin IPs fijas: funciona igual en cualquier ambiente

- Conexión: Seguimiento · Cómo se habla: El traceId viaja en la cabecera de cada llamada HTTP · Por qué así: Una sola búsqueda en Grafana sigue todo el recorrido

IDEAS CLAVE

- El navegador solo conoce la UI
- nginx reenvía /api al servicio
- El evento viaja después, desde el relevador
- Se encuentran por nombre, no por IP
- El traceId acompaña cada llamada

LO QUE SE LLEVA EL STAFF

Cada conexión tiene un porqué: cambiar una dirección o seguir un error no obliga a tocar todo.

QUÉ DECIR (? = HACER CLIC)

"Gracias. Ya vimos que los servicios no comparten base. Entonces, ¿cómo se hablan? Esta tabla lo resume, fila por fila. Primero, el navegador solo conoce la interfaz, nunca las direcciones internas."

"? La interfaz tampoco llama directo a los servicios: nginx, su servidor web, recibe todo lo que empieza con /api y lo reenvía al servicio que corresponde. Si un servicio cambia de dirección, la pantalla ni se entera."

"? Entre servicios, solicitudes le manda el evento a notificaciones con un POST en JSON. Pero no en el momento en que la persona envía, sino después, desde un proceso que llamamos el relevador. En un momento vemos por qué."

"? Nadie usa direcciones IP: cada pieza encuentra a la otra por su nombre. ? Y en cada llamada viaja un traceId, un número que nos deja seguir el recorrido completo en los logs."

SI HAY TIEMPO, SE PUEDE AGREGAR

- nginx espera menos que la interfaz, nueve segundos contra diez, para que el proxy responda con un error claro antes de que el navegador corte la petición.
- La llamada entre servicios usa un cliente HTTP preparado para pasar el traceId. Con un cliente cualquiera, el seguimiento entre servicios se rompería; por eso es una regla del equipo.

SI PREGUNTAN

¿Por qué no se llaman directamente por localhost?

Porque dentro de un contenedor localhost es el mismo contenedor; por eso cada servicio se busca por su nombre.

SI PREGUNTAN

¿Por qué HTTP y no una cola de mensajes como RabbitMQ o Kafka?

Para el laboratorio, HTTP con Outbox da la garantía que necesitamos sin otra pieza que operar. Si el volumen crece, el mismo diseño se puede pasar a una cola sin tocar las reglas.

PARA PASAR

"Hay dos formas de hablarse, y usamos las dos."

22. Síncrono o asíncrono

Presenta: Desarrollo 5 (8 personas) · Desarrollo 4 (7 personas)

EN PANTALLA · DOS MANERAS, LADO A LADO

- Síncrono: La UI llama y espera que se complete el proceso.; Respuesta inmediata; Si el servicio se cae, la persona ve error
- Asíncrono: Servicio solicitudes envía evento a servicio notificaciones y continúa con su proceso; Respuesta puede demorar; Si notificaciones se cae, el evento espera

IDEAS CLAVE

- Síncrono: la UI llama y espera
- Asíncrono: solicitudes envía el evento y sigue
- Síncrono responde ya; asíncrono puede demorar
- Si notificaciones se cae, el evento espera

Comparación: Síncrono es una llamada: me quedo en la línea hasta que me responden. Asíncrono es un correo: lo envío y sigo con lo mío.

LO QUE SE LLEVA EL STAFF

La caída de un sistema secundario no detiene a quien está atendiendo al usuario.

QUÉ DECIR (? = HACER CLIC)

"Hay dos formas de comunicarse. La síncrona es como una llamada telefónica: la interfaz llama y espera a que el proceso se complete. La respuesta es inmediata, y por eso la usamos para crear, aprobar o rechazar: la persona necesita saber el resultado en ese momento. El costo es que, si el servicio se cae, la persona ve el error."

"? La asíncrona es como un correo: el servicio de solicitudes envía el evento a notificaciones y continúa con su proceso, sin esperar. La respuesta puede demorar unos segundos, pero si notificaciones se cae, el evento no se pierde: espera hasta que el servicio vuelva."

"La regla que aprendimos: lo que la persona necesita ver, síncrono; lo que puede esperar, asíncrono. Y esto pasa también en sus sistemas de hoy: un procedimiento que llama a otro sistema y espera su respuesta falla si el otro está caído."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Un ejemplo de la vida diaria: cuando pagamos con tarjeta, la aprobación es síncrona, porque necesitamos saber si pasó; el correo con el comprobante es asíncrono, porque puede llegar un minuto después.
- La pregunta que nos hacemos en cada caso es: si el otro sistema está caído, ¿la persona tiene que enterarse? Si la respuesta es no, va asíncrono.

SI PREGUNTAN

¿Por qué no hacerlo todo asíncrono?

Porque quien crea o aprueba una solicitud necesita saber en ese momento si se pudo. Lo asíncrono es para lo que puede esperar.

PARA PASAR

"¿Y cómo logramos que el evento espere sin perderse? Con el patrón Outbox."

23. Patrón Outbox: guardar primero, avisar después

Presenta: Desarrollo 5 (8 personas) · Desarrollo 4 (7 personas)

EN PANTALLA · FLUJO PASO A PASO

- 1. Enviar: Solicitud ENVIADA + evento pendiente, en una transacción.
- 2. Bandeja de salida: El evento espera en la tabla evento.
- 3. Relevador: Cada 5 s entrega los pendientes, hasta 3 intentos.
- 4. Notificaciones: Lo recibe y el evento queda procesado.
- Si si falla en el paso 3: Sigue pendiente: Mismo eventoId; se reintenta en el siguiente ciclo.

IDEAS CLAVE

- Solicitud ENVIADA y evento pendiente, juntos
- Si una falla, no se guarda ninguna
- El relevador entrega cada 5 s, con 3 intentos
- Si falla, sigue pendiente con el mismo eventoId

Comparación: Como dejar una carta en la bandeja de salida: aunque el mensajero no pase hoy, la carta no se pierde.

LO QUE SE LLEVA EL STAFF

Ningún aviso se pierde, aunque el otro sistema esté caído. Y se puede aplicar también con procedimientos y un job.

QUÉ DECIR (? = HACER CLIC)

"Pensemos en el problema. Si al enviar una solicitud llamáramos directamente a notificaciones y justo estuviera caído, la solicitud quedaría enviada y nadie se enteraría. Por eso usamos el patrón Outbox, o bandeja de salida, que se apoya en la transacción que vimos en la parte de datos."

"Al enviar, en una sola transacción se guardan la solicitud como enviada y su aviso pendiente. O se guardan las dos cosas, o ninguna, y la persona recibe su respuesta enseguida. ? El aviso queda esperando en la tabla evento, como una carta en la bandeja de salida."

"? Después, un proceso que llamamos el relevador revisa la bandeja cada cinco segundos y entrega los pendientes, con hasta tres intentos. ? Si notificaciones lo recibe, la fila se marca como procesada. ? Si falla, la fila sigue pendiente, con el mismo número de evento, y se reintenta en el siguiente ciclo."

"Esto se puede aplicar hoy mismo en un sistema con procedimientos almacenados: en lugar de llamar a otro sistema dentro del procedimiento, se anota el aviso en una tabla de pendientes y un job lo entrega."

SI HAY TIEMPO, SE PUEDE AGREGAR

- El relevador hace hasta tres intentos por ciclo, esperando un poco más entre cada uno. Si los tres fallan, no pasa nada grave: la fila sigue pendiente y el siguiente ciclo, cinco segundos después, vuelve a intentar.
- El número del evento se genera una sola vez, al guardar. Por eso cada reintento manda exactamente el mismo número, y del otro lado se puede reconocer como repetido.
- En un sistema con procedimientos se traduce así: una tabla de pendientes con una columna procesado, un INSERT dentro de la misma transacción del procedimiento y un job que recorre los

pendientes.

SI PREGUNTAN

¿Que el envío responda bien significa que el aviso ya existe?
No. Significa que la solicitud y el evento quedaron guardados; el aviso llega después.

SI PREGUNTAN

¿Qué pasa si se cae solicitudes con avisos pendientes?
Nada se pierde: están en la base. Cuando el servicio vuelve, el relevador los encuentra y los entrega.

PARA PASAR

“Pero si se reintenta, ¿no puede llegar el aviso dos veces?”

---

24. Idempotencia: el mismo evento, un solo aviso

Presenta: Desarrollo 5 (8 personas) · Desarrollo 4 (7 personas)

EN PANTALLA · FLUJO PASO A PASO

- 1. Llega el evento: Trae su eventoId.
- 2. ¿Ya lo recibimos?: Se busca en evento_procesado.
- 3. Crear el aviso: Solo si el número es nuevo.
- 4. Anotar el número: En la misma transacción.
- Si ya estaba en el paso 2: Ignorarlo: No se crea otro aviso. Duplicados: cero.

IDEAS CLAVE

- Cada evento trae un número que no cambia
- Notificaciones anota los números recibidos
- Si llega uno repetido, lo ignora
- Se comprueba con el botón Reentregar

Comparación: Como un portero con lista de invitados: si alguien ya entró, no lo vuelve a anotar.

LO QUE SE LLEVA EL STAFF

Se puede reintentar sin miedo: un pago, un correo o un aviso nunca se duplica.

QUÉ DECIR (? = HACER CLIC)

“Sí, puede llegar dos veces. Imaginen que notificaciones recibió el evento y creó el aviso, pero la respuesta se perdió en la red. El relevador cree que falló y lo vuelve a mandar. Para eso está la idempotencia: repetir la misma operación no multiplica su efecto.”

“Así funciona: llega el evento con su número. ? Notificaciones busca ese número en la tabla evento_procesado. ? Si es nuevo, crea el aviso ? y anota el número, en la misma transacción. ? Si ya estaba, lo ignora: no se crea otro aviso. Como un portero con lista de invitados.”

“Los dos patrones se complementan: el Outbox evita perder y la idempotencia evita duplicar. Y sirve en cualquier sistema: todo proceso que se reintenta, un pago, un correo, un registro, debería tener una tabla de procesados con una llave única.”

QUÉ HACER (NO SE DICE)

- Mostrar: En la UI, pestaña Enviadas: Reentregar un evento ya consumido. Debe verse: El diálogo “El aviso no se duplicó” y Avisos duplicados en 0.

SI HAY TIEMPO, SE PUEDE AGREGAR

- Lo probamos a propósito: la interfaz tiene un botón para reentregar un evento ya consumido, y el contador de avisos duplicados se mantiene en cero.
- La idempotencia no es solo para mensajes: un pago, un registro de asistencia o una importación de archivos deberían poder reintentarse sin duplicar. La receta es la misma: un número único por operación y una tabla que lo recuerde.

SI PREGUNTAN

¿Qué resuelve el Outbox y qué resuelve la idempotencia?

El Outbox evita perder el evento; la idempotencia evita duplicar su efecto.

SI PREGUNTAN

¿Y si llegan dos entregas al mismo tiempo?

La llave primaria de evento_procesado lo impide: solo una de las dos transacciones puede anotar el número; la otra falla y su reintento lo encuentra ya procesado.

PARA PASAR

"Esto es lo que aprendimos de la interconexión."

---

25. Interconexión: lo que aprendimos

Presenta: Desarrollo 5 (8 personas) · Desarrollo 4 (7 personas)

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 0 avisos perdidos
- Métrica: 0 avisos duplicados
- Síncrono para ver, asíncrono para avisar: La UI espera la respuesta; el evento se entrega después. Valor: Una caída parcial no detiene a quien usa el sistema.
- Guardar primero, avisar después: Solicitud y evento en la misma transacción; el relevador entrega. Valor: Un servicio caído no hace perder información.
- Todo reintento necesita idempotencia: Un número de evento fijo y la lista de los ya recibidos. Valor: Reintentar sin miedo a duplicar.
- El contrato es lo único compartido: SolicitudEnviada: eventoId, id y tipo, igual en los dos lados. Valor: Equipos independientes con un acuerdo claro.

IDEAS CLAVE

- Síncrono para ver, asíncrono para avisar
- Guardar primero, avisar después
- Todo reintento necesita idempotencia
- El contrato es lo único compartido

LO QUE SE LLEVA EL STAFF

Cero avisos perdidos y cero duplicados, con patrones que sirven en cualquier tecnología.

QUÉ DECIR (? = HACER CLIC)

"Cuatro aprendizajes. ? Síncrono para lo que la persona necesita ver, asíncrono para lo que puede esperar. ? Guardar primero y avisar después: una transacción local y una bandeja de salida son más confiables que coordinar dos sistemas al mismo tiempo."

"? Todo reintento necesita idempotencia del otro lado. ? Y entre dos servicios, lo único compartido es el contrato del mensaje; si cambia, cambia en los dos lados."

SI HAY TIEMPO, SE PUEDE AGREGAR

- Juntos, Outbox e idempotencia logran lo que en la industria se llama entrega al menos una vez con efecto exactamente una vez. Es la base de muchos sistemas de pagos y de notificaciones.

PARA PASAR

“Hasta aquí, el código y los datos. Ahora, ¿cómo llevamos todo esto a cualquier ambiente? Le paso la palabra a infraestructura.”

5 · DevOps: Docker y Kubernetes

Docker empaqueta, Kubernetes administra y el pipeline prueba cada cambio.

26. Docker y Kubernetes: no compiten, se complementan

Presenta: Infraestructura 1

EN PANTALLA · DOS MANERAS, LADO A LADO

- Docker: empaqueta: Mete la app con Java y todo lo que usa; Crea una imagen que no cambia; Corre igual en una laptop o en un servidor; Compose levanta varias cajas juntas
- Kubernetes: administra: Decide dónde corre cada caja; Si una se cae, levanta otra; Le da a cada servicio un nombre fijo; Puede correr más copias si hay carga

IDEAS CLAVE

- Docker empaqueta la app con todo lo que necesita
- Kubernetes decide dónde corre y la vigila
- La misma imagen sirve en los dos
- Uno sin el otro queda a medias

Comparación: Como el transporte de carga: Docker es el contenedor estándar y Kubernetes, el puerto que los acomoda, los vigila y reemplaza el que se daña.

LO QUE SE LLEVA EL STAFF

El mismo paquete del laptop al servidor, y un sistema que se cuida solo.

QUÉ DECIR (? = HACER CLIC)

“Gracias. Docker y Kubernetes muchas veces se presentan como si uno reemplazara al otro, y no es así: no compiten, se complementan. Docker resuelve el clásico “en mi máquina funciona”: empaqueta la aplicación con Java y todo lo que necesita en una imagen, un paquete que no cambia y que corre igual en una laptop o en un servidor.”

“? Kubernetes empieza donde termina Docker. No empaqueta nada: toma esas imágenes y las administra. Decide dónde corre cada contenedor, lo vigila y, si uno se cae, levanta otro.”

“Piénsenlo como el transporte de carga: Docker es el contenedor estándar y Kubernetes es el puerto que los acomoda y reemplaza el que se daña. En nuestra computadora, Docker Compose levanta los ocho contenedores del sistema con un solo comando; en el clúster, esas mismas imágenes las administra Kubernetes, sin recompilar nada.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- La imagen de cada servicio se construye en dos etapas: una con Maven para compilar y otra solo con Java para correr. Así la imagen final es más liviana y no lleva herramientas de compilación.

- Además, el contenedor corre con un usuario sin privilegios: si alguien comprometiera la aplicación, no sería administrador del contenedor.

SI PREGUNTAN

¿Docker es una máquina virtual?

No. Es más liviano: comparte el sistema operativo y solo aísla la aplicación y lo que necesita.

SI PREGUNTAN

¿Necesitamos Kubernetes para todo?

No. Para un sistema pequeño basta Docker Compose en un servidor. Kubernetes vale la pena cuando hay varios servicios, varias copias o hay que recuperarse solo.

PARA PASAR

"Primero, cómo cada cambio se convierte en una imagen: el pipeline."

---

27. CI/CD: del push a la imagen publicada

Presenta: Infraestructura 1

EN PANTALLA · DOS CAMINOS, LADO A LADO

- Pull Request hacia main: Abres el PR: Desde tu rama feat/… -> Compila y prueba: mvn verify con PostgreSQL temporal -> Arranca: El servicio responde de verdad -> Construye la imagen: docker build -> Verde o rojo: El resultado queda en el PR (CI · integración continua: pasos 2 a 4)
- Fusión a main: Fusionas a main: Después de la revisión -> El mismo CI: Compila, prueba y arranca -> Publica en GHCR: La imagen, con el SHA del commit -> Despliegue: kubectl apply -f k8s/ · manual (CI: pasos 2 a 2; CD · entrega continua: pasos 3 a 3; Hoy, a mano: pasos 4 a 4)

IDEAS CLAVE

- Un push a una rama de trabajo, solo, no dispara nada
- Pull Request: el CI compila, prueba, arranca y construye
- Fusión a main: el mismo CI y luego el CD publica la imagen
- Desplegar en Kubernetes hoy es un paso manual

LO QUE SE LLEVA EL STAFF

Un error se detecta antes de llegar a main, y cada imagen se rastrea hasta su commit.

QUÉ DECIR (? = HACER CLIC)

"Así funciona el pipeline, desde el momento del push. Primero, algo que sorprende: un push a una rama de trabajo, por sí solo, no dispara nada. El pipeline arranca cuando se abre o se actualiza un Pull Request hacia main. Ahí corre la integración continua, el CI: un servidor de GitHub compila cada servicio, corre todas sus pruebas contra un PostgreSQL temporal, arranca el servicio de verdad hasta ver sus datos iniciales y construye la imagen Docker. El resultado queda en el mismo Pull Request: verde o rojo."

"? Cuando el Pull Request se aprueba y se fusiona a main, corre otra vez el mismo CI, y después viene el CD, la entrega continua: la imagen se publica en el registro de GitHub, etiquetada con el código del commit, lista para desplegar."

"Y una distinción que vale la pena: CD puede ser entrega continua, que es dejar la imagen lista, o despliegue continuo, que es instalarla sola en el ambiente. Nosotros llegamos a la entrega; el despliegue en Kubernetes es un paso aparte, con kubectl apply. Automatizarlo es el siguiente paso natural, con una aprobación de por medio."

SI HAY TIEMPO, SE PUEDE AGREGAR

- El pipeline prueba los dos servicios en paralelo, cada uno con su propia base temporal, y tiene un trabajo aparte para la interfaz: valida, prueba, construye y comprueba que el archivo de Compose es válido.
- La imagen de la interfaz solo se publica cuando el cambio llega a main; la de los servicios hoy también se publica desde el Pull Request. Es un ajuste que ya tenemos identificado.
- Cada imagen lleva como etiqueta el código del commit. Si algo falla en un ambiente, sabemos exactamente qué versión está corriendo y podemos volver a la anterior.

SI PREGUNTAN

¿Por qué no se despliega solo?

Porque es un laboratorio con un clúster local. En un ambiente real se agrega un paso de despliegue con aprobación, o una herramienta que aplica en el clúster lo que está en main.

SI PREGUNTAN

¿Las pruebas usan la base real?

No: GitHub crea un PostgreSQL temporal para cada ejecución, con una clave distinta cada vez, y lo descarta al terminar.

PARA PASAR

“Y para probar o mostrar todo esto sin instalar nada, usamos Codespaces.”

---

28. Sin instalar nada: GitHub Codespaces

Presenta: Infraestructura 1

EN PANTALLA · FLUJO PASO A PASO

- 1. Una rama: Code ? Codespaces ? Crear.
- 2. Máquina en la nube: 4 núcleos y Docker adentro.
- 3. levantar.sh: Construye y levanta los 8 contenedores.
- 4. Puertos https: UI, Grafana y Swagger, privados.
- 5. Demo: Desde el navegador.

IDEAS CLAVE

- Se crea desde una rama del repositorio
- Una máquina en la nube con Docker adentro
- Un script construye y levanta los 8 contenedores
- La UI, Grafana y Swagger se abren por https

LO QUE SE LLEVA EL STAFF

Cualquiera del equipo prueba o muestra el sistema desde un navegador, sin instalar nada.

QUÉ DECIR (? = HACER CLIC)

“Una pieza que nos resolvió la demostración: GitHub Codespaces. Desde una rama del repositorio se crea una máquina en la nube lista para trabajar. ? La configuración vive en el mismo repositorio, en la carpeta .devcontainer: pide cuatro núcleos y Docker adentro.”

“? Al crearla, un script construye las imágenes y levanta los ocho contenedores, igual que en nuestra computadora. ? Cada puerto queda publicado por https y privado para la cuenta que lo abrió: la interfaz, Grafana y los dos Swagger. ? Y la demostración se hace desde el navegador.”

“El valor: cualquiera del equipo puede probar o mostrar el sistema sin instalar nada y sin depender de la red de la oficina. Lo verificamos con la prueba de humo completa: siete de siete.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- La máquina publica cuatro puertos: la interfaz, Grafana y los dos Swagger. Un script escribe la dirección pública de cada puerto para que los enlaces de la interfaz también funcionen en la nube.
- Un Codespace sin uso se detiene solo a los treinta minutos, y se borra al terminar. Para el día de la presentación lo arrancamos quince minutos antes.

SI PREGUNTAN

¿Cuánto cuesta?

GitHub lo cobra por horas de núcleo: esta máquina de 4 núcleos consume 4 por cada hora encendida. Se detiene al terminar y deja de consumir.

PARA PASAR

“Ahora, el clúster: qué piezas de Kubernetes usamos. Le paso la palabra a mi compañero.”

---

29. Los recursos de Kubernetes

Presenta: Infraestructura 2

EN PANTALLA · CAPAS DEL SISTEMA

- Acceso: Service: Un nombre fijo por componente; port-forward: La UI en localhost:8088
- Cómputo: 5 Deployments: UI, 2 servicios y 2 bases; initContainers: Esperan a la base y al otro servicio; readinessProbe: Sin tráfico hasta estar listo
- Configuración: Secret arka-postgres: La clave de las bases; Variables de entorno: DB_URL, NOTIFICACIONES_URL
- Datos: PersistentVolumeClaim: Un disco por base: sobrevive al Pod

IDEAS CLAVE

- Deployment y Pod por componente
- Service: un nombre fijo
- Secret para la clave de la base
- Un disco persistente para cada base

LO QUE SE LLEVA EL STAFF

El estado del sistema queda escrito y versionado, no en la memoria de alguien.

QUÉ DECIR (? = HACER CLIC)

“Gracias. En Kubernetes no damos órdenes paso a paso: describimos en archivos YAML cómo queremos que esté el sistema, y Kubernetes lo mantiene así. Arriba, el acceso: un Service le da a cada componente un nombre fijo, para que los demás lo encuentren aunque el contenedor cambie.”

“? Luego, el cómputo: un Deployment por componente, cinco en total, que dice cuántas copias queremos. Antes de arrancar, cada servicio espera a que su base responda, y una prueba de salud evita mandarle tráfico antes de tiempo.”

“? La configuración: la clave de la base en un Secret, como vimos en la parte de datos, y el resto en variables de entorno. ? Y los datos: cada base tiene su disco persistente, que sobrevive aunque el Pod se recree. Todo se aplica con un solo comando: kubectl apply sobre la carpeta k8s.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Antes de arrancar cada servicio, un contenedor previo espera a que su base responda. Sin eso, el servicio arrancaría, no encontraría la base y se reiniciaría una y otra vez.
- La prueba de salud le dice a Kubernetes cuándo el servicio está listo para recibir tráfico. Arrancar no es lo mismo que estar listo, y esa diferencia evita errores en los primeros segundos.

SI PREGUNTAN

¿Por qué la base corre como un Deployment y no como un servicio administrado?
Porque es un ambiente de laboratorio. En producción conviene una base administrada o un StatefulSet con respaldos.

PARA PASAR

“¿Y qué pasa cuando algo se cae?”

---

30. Cuando un Pod se cae

Presenta: Infraestructura 2

EN PANTALLA · QUÉ PASA CUANDO FALLA

- Falla: Se cae el Pod de solicitudes: Un error, un nodo reiniciado o un kubectl delete.
- 1. [0 s] Detecta: Falta 1 de 1 copias.
- 2. [+1 s] Recrea: Un Pod nuevo con la misma imagen.
- 3. [+segundos] Espera: initContainers: base y notificaciones.
- 4. [Ready] Listo: La readinessProbe responde.
- Resultado: Sin intervención manual: Los eventos pendientes siguen en la base y se entregan.

IDEAS CLAVE

- El Deployment detecta que falta una copia
- Crea un Pod nuevo
- Espera a la base y a su salud
- Vuelve a recibir tráfico, sin intervención

LO QUE SE LLEVA EL STAFF

Un sistema que se recupera solo, sin que nadie se despierte a las tres de la mañana.

QUÉ DECIR (? = HACER CLIC)

“Kubernetes funciona como un termostato: no le decimos qué hacer paso a paso, le decimos el estado que queremos, por ejemplo una copia de cada servicio, y él lo mantiene. Veamos qué pasa si se cae el Pod de solicitudes.”

“? En el mismo instante, el Deployment detecta que falta una copia. ? En un segundo, crea un Pod nuevo con la misma imagen. ? El Pod nuevo espera a que su base y notificaciones respondan, ? y solo cuando su prueba de salud contesta, vuelve a recibir tráfico.”

“? Nadie intervino. Y como los avisos pendientes están en la base, no en memoria, nada se pierde mientras tanto. Aquí se juntan las tres partes que vimos a fondo: la base guarda, el Outbox espera y Kubernetes repara.”

QUÉ HACER (NO SE DICE)

- Mostrar: kubectl delete pod -l app=servicio-solicitudes y luego kubectl get pods -w. Debe verse: Aparece un Pod nuevo que vuelve a estar Ready.

SI HAY TIEMPO, SE PUEDE AGREGAR

- Se demuestra en un minuto: se borra el Pod con kubectl delete y, mirando la lista de Pods, aparece uno nuevo que en unos segundos vuelve a estar listo.
- Si lo que se cae es la base, pasa lo mismo: su Pod se recrea y los datos siguen en el disco persistente.

SI PREGUNTAN

¿Y si se cae la base?
Su Pod también se recrea y los datos siguen en el disco persistente; los servicios esperan a que vuelva.

PARA PASAR

“Esto es lo que aprendimos de DevOps.”

---

31. DevOps: lo que aprendimos

Presenta: Infraestructura 2

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 1 comando para levantar todo
- Métrica: 1 imagen por commit, rastreable
- Docker y Kubernetes se complementan: Uno empaqueta; el otro ejecuta y cuida. Valor: Un mismo paquete, de la laptop al clúster.
- La imagen es el entregable: Lo que prueba el pipeline es lo que se despliega. Valor: Sin sorpresas entre ambientes.
- Configurar por variables: La cadena de conexión y las direcciones vienen del ambiente. Valor: Un solo artefacto para todos los ambientes.
- Declarar el estado deseado: Deployments, pruebas de salud y un pipeline que arranca el servicio. Valor: El clúster se repara solo y los errores aparecen antes.

IDEAS CLAVE

- Docker y Kubernetes se complementan
- La imagen es el entregable
- Configurar por variables, no por código

- Declarar el estado y probar que arranca

LO QUE SE LLEVA EL STAFF

Despliegues repetibles y un sistema que se repara solo.

QUÉ DECIR (? = HACER CLIC)

“Cuatro aprendizajes de DevOps. ? Docker y Kubernetes no compiten: uno empaqueta y el otro ejecuta y cuida. ? La imagen es el entregable: lo que prueba el pipeline es exactamente lo que se despliega.”

“? Configurar por variables y no por código, empezando por la cadena de conexión. ? Y declarar el estado que queremos, y probar que el servicio arranca, no solo que compila.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Lo que más valor nos dio de DevOps fue que todo quedó escrito: la receta de cada imagen, el archivo de Compose, los manifiestos de Kubernetes y el pipeline. Nada depende de la memoria de una persona.

PARA PASAR

“Ya está desplegado. ¿Cómo sabemos qué pasa adentro?”

6 · Observabilidad

Logs en un solo lugar, trazas entre servicios y salud de cada componente.

32. Todos los logs, en un solo lugar

Presenta: Infraestructura 2

EN PANTALLA · FLUJO PASO A PASO

- 1. Servicios: Logs JSON con traceId y eventoId.
- 2. Promtail: Los recoge de cada contenedor.
- 3. Loki: Los guarda, etiquetados por servicio.
- 4. Grafana: Una búsqueda para todo el sistema.

IDEAS CLAVE

- Cada servicio escribe sus logs en JSON
- Promtail los recoge de todos los contenedores
- Loki los guarda por servicio
- Grafana los busca por traceId o eventoId

Comparación: Como la caja negra de un avión: registra lo que pasó, para revisarlo después.

LO QUE SE LLEVA EL STAFF

Diagnosticar un error entre sistemas en minutos, con una sola búsqueda.

QUÉ DECIR (? = HACER CLIC)

“La observabilidad la vemos de forma general. Con varios servicios, cada uno tiene sus logs, y si algo falla no queremos entrar contenedor por contenedor. Cada servicio escribe sus logs en JSON, con un formato estándar. ? Promtail los recoge de todos los contenedores, ? Loki los guarda etiquetados por servicio ? y Grafana nos deja buscarlos en un solo lugar.”

“Como cada llamada lleva su traceId, con una sola búsqueda vemos el viaje completo de un evento: cuándo lo publicó el relevador y cuándo lo consumió notificaciones. Y cada servicio expone su salud, que la interfaz muestra en verde o en rojo; lo van a ver en la demostración.”

QUÉ HACER (NO SE DICE)

- Mostrar: Grafana ? Explore ? Loki: {service="servicio-notificaciones"} |= "evento". Debe verse: Las líneas “Consume evento” y “aviso creado” con el número del evento.

SI HAY TIEMPO, SE PUEDE AGREGAR

- Para buscar en Grafana basta una consulta como: los logs del servicio de notificaciones que contengan la palabra evento. En segundos aparecen las líneas con el número del evento.
- Los logs salen en un formato estándar, ECS, sin tocar el código del negocio: es una línea de configuración de Spring Boot.

PARA PASAR

“Esto es lo que aprendimos de observar el sistema.”

---

33. Observabilidad: lo que aprendimos

Presenta: Infraestructura 2

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 1 lugar para buscar todos los logs
- Métrica: 8 contenedores observados
- Logs estructurados desde el día uno: JSON con formato estándar, sin cambiar el código de negocio. Valor: Buscar por campo en lugar de leer texto.
- Un identificador que cruce servicios: El traceId viaja en cada llamada y el eventoId en cada log. Valor: Diagnosticar un fallo entre servicios en minutos.
- Salud no es lo mismo que correcto: UP dice que responde; los pendientes dicen si entrega. Valor: Medir el resultado del negocio, no solo el proceso.

IDEAS CLAVE

- Logs estructurados desde el día uno
- Un identificador que cruce servicios
- Salud no es lo mismo que correcto

LO QUE SE LLEVA EL STAFF

Saber qué pasa adentro sin adivinar.

QUÉ DECIR (? = HACER CLIC)

“Tres aprendizajes. ? Los logs estructurados se configuran desde el día uno; después es mucho más caro. ? En un sistema distribuido hace falta un identificador que cruce los servicios. ? Y salud no es lo mismo que correcto: un servicio puede estar arriba y tener avisos pendientes; hay que medir el resultado, no solo el proceso.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Si tuvieran que empezar por una sola cosa, sería un identificador por operación que viaje entre sistemas y aparezca en cada log. Con eso, diagnosticar deja de ser adivinar.

PARA PASAR

“Y ahora sí, veamos todo esto funcionando.”

7 · Demostración con la UI

La UI ejemplifica en vivo lo que contamos: estados, Outbox, idempotencia y salud.

34. La UI: lo que contamos, en vivo

Presenta: Desarrollo 4 (8 personas) · Desarrollo 3 (7 personas)

EN PANTALLA · CAPTURAS REALES

- Captura de Solicitudes (localhost:8088/#/solicitudes): El negocio: estados y errores 400, 404 y 409.
- Captura de Enviadas (localhost:8088/#/enviadas): La interconexión: pendientes, entregados y Reentregar.
- Captura de Salud (localhost:8088/#/salud): La observabilidad: UP o DOWN, Swagger y Grafana.
- Captura de Ayuda (localhost:8088/#/ayuda): Diagramas paso a paso.

IDEAS CLAVE

- Solicitudes: los estados y los errores claros
- Enviadas: la bandeja de salida y Reentregar
- Salud: cada servicio, Swagger y Grafana
- Ayuda: diagramas de cómo funciona

LO QUE SE LLEVA EL STAFF

Ver lo invisible: la bandeja de salida, los reintentos y la idempotencia, en pantalla.

QUÉ DECIR (? = HACER CLIC)

“Gracias. Todo lo que les contamos se puede ver funcionando, y para eso construimos esta interfaz. No es el entregable del proyecto: es la forma de ejemplificar cada parte en vivo. En Solicitudes se ve el negocio: crear, enviar, aprobar o rechazar, con sus mensajes de error claros.”

“? En Enviadas se ve la interconexión: la bandeja de salida, qué eventos están pendientes, cuáles se entregaron y un botón para reentregar a propósito. ? En Salud, si cada servicio está arriba, con enlaces a Swagger y a Grafana. ? Y en Ayuda, diagramas de cómo funciona.”

“Hagamos la prueba más interesante: tumbar un servicio a propósito.”

QUÉ HACER (NO SE DICE)

- Cambiar al navegador con la UI: http://localhost:8088 o el puerto 8088 del Codespace. Los comandos docker compose de la siguiente lámina se escriben en esa misma computadora o en la terminal del Codespace. Si el sistema no está disponible, seguir con las capturas de esta lámina y de la siguiente.

SI HAY TIEMPO, SE PUEDE AGREGAR

- La interfaz es una aplicación React servida por nginx, que a la vez hace de proxy hacia los servicios. Tiene sus propias pruebas automáticas y corre en el mismo Compose que todo lo demás.
- La pestaña Ayuda tiene diagramas paso a paso de cada flujo; la usamos para explicar el sistema a quien no lo conoce.

PARA PASAR

"Vamos a detener el servicio de notificaciones."

---

35. En vivo: si notificaciones se cae

Presenta: Desarrollo 4 (8 personas) · Desarrollo 3 (7 personas)

EN PANTALLA · QUÉ PASA CUANDO FALLA

- Falla: notificaciones está caído: docker compose stop servicio-notificaciones
- 1. [0 s] Enviar: Responde bien: solicitud ENVIADA y evento guardado.
- 2. [cada 5 s] Reintenta: 3 intentos por ciclo; el evento sigue pendiente.
- 3. [en la UI] Se ve: "Notificaciones no responde" y 1 en camino.
- 4. [al volver] Entrega: Mismo eventoId: se crea el aviso.
- Resultado: 1 aviso, 0 perdidos: Sin intervención y, al reentregar, sin duplicados.

IDEAS CLAVE

- Enviar responde bien aunque notificaciones esté caído
- El evento queda pendiente en la bandeja
- El relevador reintenta cada 5 segundos
- Al volver, llega un solo aviso

LO QUE SE LLEVA EL STAFF

Ver con sus propios ojos que una caída no hace perder ni duplicar información.

QUÉ DECIR (? = HACER CLIC)

"Detenemos notificaciones a propósito; en Salud aparece en rojo. ? Ahora enviamos una solicitud. La respuesta es buena: la solicitud queda enviada y su evento queda guardado. La persona no ve ningún error, porque enviar no depende del otro servicio. Eso es lo asíncrono."

"? Mientras tanto, el relevador intenta entregar el evento cada cinco segundos y falla, pero no lo pierde. ? La pantalla de Enviadas lo muestra: notificaciones no responde, y hay un evento en camino."

"? Levantamos notificaciones, y en el siguiente ciclo el evento se entrega con el mismo número: aparece exactamente un aviso. Eso es el Outbox. ? Y para cerrar, reentregamos ese mismo evento a propósito: notificaciones lo reconoce y no crea otro aviso. Cero perdidos y cero duplicados, en vivo."

QUÉ HACER (NO SE DICE)

- Antes: docker compose stop servicio-notificaciones. Mostrar Salud en rojo (DOWN).
- Crear y enviar DEMO-01; mostrar en Enviadas el aviso amarillo "Notificaciones no responde" y "1 en

camino”. No hace falta recargar: la pantalla se actualiza sola.

- docker compose start servicio-notificaciones y esperar unos segundos: el evento pasa a entregado y aparece 1 aviso.
- En Enviadas, pulsar Reentregar sobre ese evento: aparece “El aviso no se duplicó” y Avisos duplicados sigue en 0.
- Los clics de la lámina acompañan lo que pasa en la UI; si la demostración en vivo falla, contarla con la lámina.
- Mostrar: Detener notificaciones, enviar DEMO-01, mirar Enviadas, volver a iniciar notificaciones y reentregar el evento. Debe verse: Evento en camino mientras está caído; al volver, entregado y 1 aviso; al reentregar, 0 duplicados.

SI HAY TIEMPO, SE PUEDE AGREGAR

- Si la red de la sala falla, la demostración se puede contar con esta misma lámina: cada clic muestra lo que se vería en la pantalla.
- Si una operación que cambia datos tarda demasiado, la interfaz no la repite sola, porque el cambio pudo haberse guardado aunque la respuesta no llegara.

SI PREGUNTAN

¿Cuánto tiempo puede esperar un evento pendiente?
Indefinidamente: está en la base. Se entrega en el primer ciclo después de que notificaciones vuelve.

PARA PASAR

“Esto es lo que aprendimos de la demostración.”

---

36. Demostración: lo que aprendimos

Presenta: Desarrollo 4 (8 personas) · Desarrollo 3 (7 personas)

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 5 fallas con reacción prevista
- Métrica: 4 pantallas para ver el sistema por dentro
- Diseñar para la falla: Cada falla probada tiene una reacción y un mensaje. Valor: Menos incidentes y menos llamadas a soporte.
- Hacer visible lo asíncrono: Pendientes, entregados y consumidos en una pantalla. Valor: Explicar el sistema sin abrir el código.
- Mostrar en vivo explica más: Botones para provocar fallas y ver la reacción. Valor: Material de capacitación para quien llegue al equipo.

IDEAS CLAVE

- Diseñar para la falla, no solo para el éxito
- Hacer visible lo asíncrono
- Mostrar en vivo explica más que una lámina

LO QUE SE LLEVA EL STAFF

Una herramienta que explica el sistema a cualquiera, también a quien llegue nuevo al equipo.

QUÉ DECIR (? = HACER CLIC)

“Tres aprendizajes de la demostración. ? Hay que diseñar para la falla, no solo para el camino feliz. ? Lo asíncrono hay que hacerlo visible: sin la pestaña Enviadas, un evento pendiente sería invisible. ? Y mostrarlo en vivo explica más que cualquier lámina; esta misma interfaz sirve para capacitar a quien llegue al equipo.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- La demostración también nos sirvió como prueba: cada falla que mostramos la provocamos antes, a propósito, varias veces.

PARA PASAR

“Para cerrar, el valor que deja todo esto.”

Valor y cierre

37. El valor agregado

Presenta: Orquestador

EN PANTALLA · TABLA DE TRES COLUMNAS

- Pilar: Datos consistentes · Qué construimos: Transacción local, Outbox e idempotencia · Valor para el equipo: 0 avisos perdidos o duplicados ante caídas
- Pilar: Conexión segura · Qué construimos: Cadena de conexión y claves fuera del código · Valor para el equipo: Cambiar de servidor o de clave sin recompilar
- Pilar: Reproducibilidad · Qué construimos: Docker, Compose y Kubernetes · Valor para el equipo: 1 comando y la misma imagen en todo ambiente
- Pilar: Calidad · Qué construimos: Pruebas automáticas y pipeline en cada cambio · Valor para el equipo: Errores detectados en minutos, antes de integrar
- Pilar: Trazabilidad · Qué construimos: Logs centralizados, traceId e imagen por commit · Valor para el equipo: Diagnóstico en minutos y versiones rastreables

IDEAS CLAVE

- Datos consistentes ante fallas
- Conexiones fuera del código
- Entornos reproducibles
- Calidad en cada cambio
- Trazabilidad de punta a punta

LO QUE SE LLEVA EL STAFF

Cinco pilares que cualquier equipo puede adoptar.

QUÉ DECIR (? = HACER CLIC)

“Gracias. Si tuviéramos que resumir el valor en una lámina, sería esta. Datos consistentes: ante una caída, no se pierde ni se duplica un aviso. ? Conexión segura: la cadena de conexión y las claves viven fuera del código.”

“? Reproducibilidad: un comando y la misma imagen en cualquier ambiente. ? Calidad: pruebas automáticas en cada cambio, antes de integrarlo. ? Y trazabilidad: logs en un solo lugar e imágenes rastreables hasta su commit.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Si alguien pregunta por números: cero avisos perdidos o duplicados en las pruebas de caída, un comando para levantar los ocho contenedores y una imagen rastreable por cada commit.

PARA PASAR

“Y lo más importante: qué pueden aplicar ustedes desde mañana.”

---

38. Lo que pueden aplicar desde mañana

Presenta: Orquestador

EN PANTALLA · TABLA DE TRES COLUMNAS

- Práctica: Cadena de conexión fuera del código · Cómo se ve en sus sistemas de hoy: Variables de entorno o un gestor de secretos en el servidor · Qué evita: Claves en el repositorio y recompilar por cambiar de servidor
- Práctica: Guardar y avisar en la misma transacción · Cómo se ve en sus sistemas de hoy: Una tabla de pendientes que llena el procedimiento y un job que la entrega · Qué evita: Avisos perdidos cuando el otro sistema está caído
- Práctica: Idempotencia en lo que se reintenta · Cómo se ve en sus sistemas de hoy: Una tabla de procesados con llave única por número de operación · Qué evita: Pagos, correos o registros duplicados
- Práctica: Un identificador que cruce sistemas · Cómo se ve en sus sistemas de hoy: El mismo número en el log de la aplicación, del procedimiento y del otro sistema · Qué evita: Horas buscando un error entre sistemas
- Práctica: El mismo paquete en todos los ambientes · Cómo se ve en sus sistemas de hoy: Contenedores para aplicaciones y bases de prueba · Qué evita: El clásico “en mi máquina funciona”

IDEAS CLAVE

- Sacar la cadena de conexión del código
- Guardar y avisar en la misma transacción
- Hacer idempotente lo que se reintenta
- Un identificador que cruce sistemas
- El mismo paquete en todos los ambientes

LO QUE SE LLEVA EL STAFF

Cinco prácticas concretas que se pueden empezar a aplicar sin cambiar de tecnología.

QUÉ DECIR (? = HACER CLIC)

“Esta es la lámina que más nos importa que se lleven. Nada de esto exige reescribir sus sistemas; todo funciona también con procedimientos almacenados. Primero: saquen la cadena de conexión del código; usen variables de entorno o un gestor de secretos.”

“? Segundo: si un procedimiento tiene que avisarle a otro sistema, no lo llamen ahí mismo. Anoten el aviso en una tabla de pendientes, en la misma transacción, y que un job lo entregue. Ese es el Outbox. ? Tercero: todo lo que se reintenta, un pago, un correo, un registro, debe ser idempotente: una tabla de procesados con llave única.”

“? Cuarto: un mismo identificador en todos los logs, para seguir una operación de un sistema a otro. ? Y quinto: el mismo paquete en todos los ambientes, para que lo que se prueba sea lo que se despliega.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- El orden que recomendamos: primero la cadena de conexión, porque es un cambio pequeño que baja un riesgo real; después la tabla de pendientes en el proceso que más falla hoy; y luego la idempotencia

en lo que se reintenta.
- Ninguna de estas prácticas exige Java, microservicios ni Kubernetes. Son decisiones de diseño que se pueden tomar en el sistema que ya tienen.

SI PREGUNTAN

¿Por dónde empezamos?

Por la cadena de conexión: es el cambio más pequeño y baja un riesgo real de seguridad. Después, la tabla de pendientes en el proceso que más falle hoy.

PARA PASAR

“Si nos llevamos solo tres ideas, que sean estas.”

---

39. Lo que nos llevamos

Presenta: Orquestador

EN PANTALLA · LO QUE APRENDIMOS

- Métrica: 0 avisos perdidos o duplicados
- Métrica: 1 comando para levantar todo
- Diseñar para la falla: Guardar primero, reintentar e ignorar duplicados. Valor: Sistemas que se recuperan solos.
- Automatizar la confianza: Pruebas, pipeline, imágenes y estado declarado. Valor: Cambiar rápido sin miedo a romper.
- Hacer visible el sistema: Logs, trazas, salud, Swagger y la UI. Valor: Diagnosticar y explicar en minutos.

IDEAS CLAVE

- Diseñar para la falla
- Automatizar la confianza
- Hacer visible el sistema

LO QUE SE LLEVA EL STAFF

Tres ideas para recordar al salir de la sala.

QUÉ DECIR (? = HACER CLIC)

“Tres ideas para cerrar. ? Diseñar para la falla: en un sistema distribuido algo siempre va a fallar, y la diferencia está en cómo reacciona. ? Automatizar la confianza: pruebas, pipeline e infraestructura declarada para cambiar rápido sin miedo. ? Y hacer visible el sistema: lo que no se ve no se puede diagnosticar ni explicar.”

SI HAY TIEMPO, SE PUEDE AGREGAR

- Si se llevan una sola frase: diseñen pensando en qué pasa cuando algo falla, no solo cuando todo sale bien. Y antes de pensar en el cómo, tengan claro el qué.

PARA PASAR

“Gracias por su atención.”

---

40. Gracias

Presenta: Orquestador

EN PANTALLA

- Equipo arka-litos
- Proyecto final · Upskilling Backend Java

QUÉ DECIR (? = HACER CLIC)

“Empezamos con un endpoint y terminamos con dos microservicios con sus bases de datos, empaquetados, desplegados, observados y documentados, que saben comportarse cuando algo falla.”

“Muchas gracias por acompañarnos, de parte de todo el equipo arka-litos. Con gusto respondemos sus preguntas.”

QUÉ HACER (NO SE DICE)

- Quien orquesta recibe las preguntas y se las pasa a quien presentó esa parte.